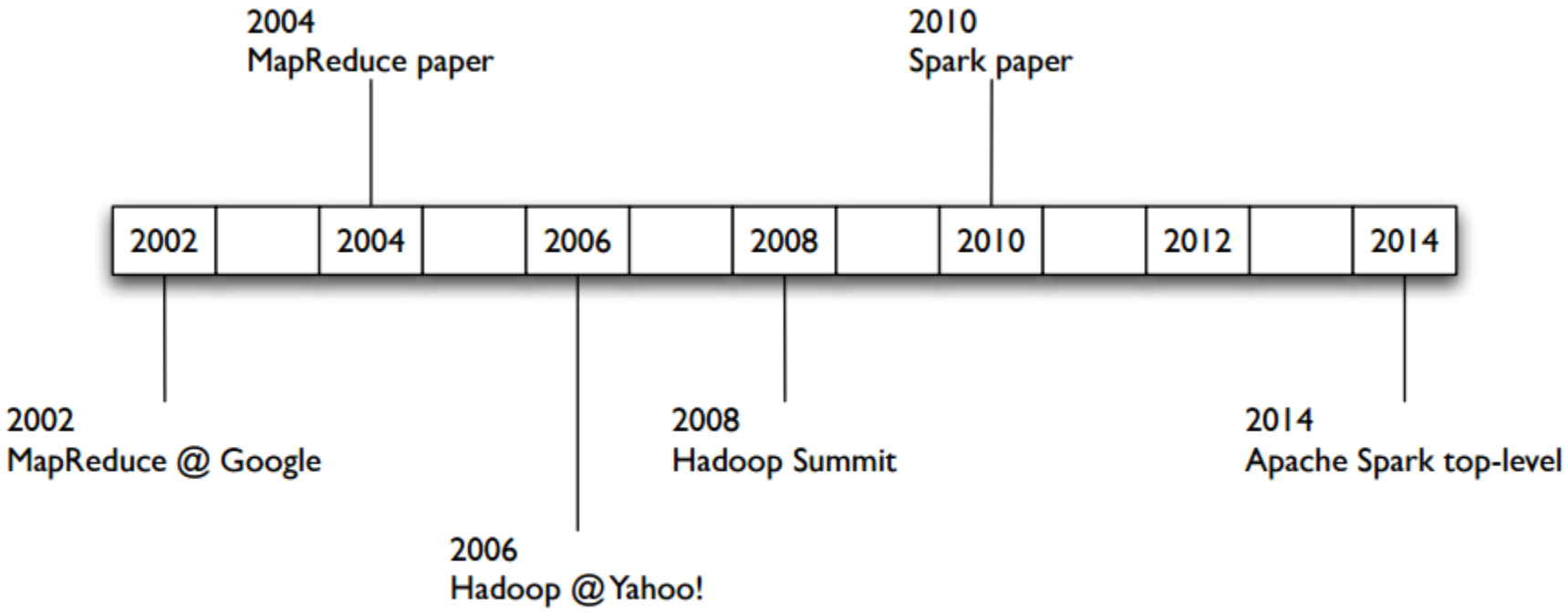
Spark, SparkSQL, Dataframes

Gombos Gergő

Történelem


2004
MapReduce paper
2010
Spark paper
2002
2004
2006
2008
2010
2012
2014
2002
MapReduce @ Google
2006
Hadoop @Yahoo!
2008
Hadoop Summit
2014
Apache Spark top-level

Spark Motiváció

- Hadoop MapReduce problémák:
  - Programozás nehézkes
  - Performancia, sok I/O használat
  - Speciális alkalmazásokra nem alkalmas

MapReduce programozás

- Hadoop: 61 sor
- Spark: 1 sor

```
sc.textFile('...').flatMap(lambda x: x.split())
    .map(lambda x: (x, 1))
    .reduceByKey(lambda x, y: x+y)
    .saveAsTextFile('...')
```

```java
import java.io.IOException;
import java.util.StringTokenizer;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCount {

  public static class TokenizerMapper
       extends Mapper<Object, Text, Text, IntWritable>{

    private final static IntWritable one = new IntWritable(1);
    private Text word = new Text();

    public void map(Object key, Text value, Context context
                    ) throws IOException, InterruptedException {
      StringTokenizer itr = new StringTokenizer(value.toString());
      while (itr.hasMoreTokens()) {
        word.set(itr.nextToken());
        context.write(word, one);
      }
    }
  }

  public static class IntSumReducer
       extends Reducer<Text,IntWritable,Text,IntWritable> {
    private IntWritable result = new IntWritable();

    public void reduce(Text key, Iterable<IntWritable> values,
                       Context context
                       ) throws IOException, InterruptedException {
      int sum = 0;
      for (IntWritable val : values) {
        sum += val.get();
      }
      result.set(sum);
      context.write(key, result);
    }
  }

  public static void main(String[] args) throws Exception {
    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "word count");
    job.setJarByClass(WordCount.class);
    job.setMapperClass(TokenizerMapper.class);
    job.setCombinerClass(IntSumReducer.class);
    job.setReducerClass(IntSumReducer.class);
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);
    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));
    System.exit(job.waitForCompletion(true) ? 0 : 1);
  }
}
```

Performancia

- Hadoop adat kiírása a diskre egy Jobban: 3 vagy több

- Hive (SQL tool Hadoopra)
  - Egy tipikus lekérdezés 3-5 MR Job
  - Mindegyik olvassa a HDD-t 3 vagy több alkalommal
  - Mindegyik ír a HDD-re minden olvasás után
  - Összesen 18-30 olvasás egy egyszerű lekérdezésnél

Performancia

- Mit ad a Spark?
  - Lusta kiértékelés
    - Optimalizálja a job-ot mielőtt lefuttatná
  - Memória alapú cache
    - Egyszer olvassa a HDD-t, utána Ram-ot használ
  - Hatékony pipeline
    - Adatok átadása a folyamatok között HDD nélkül

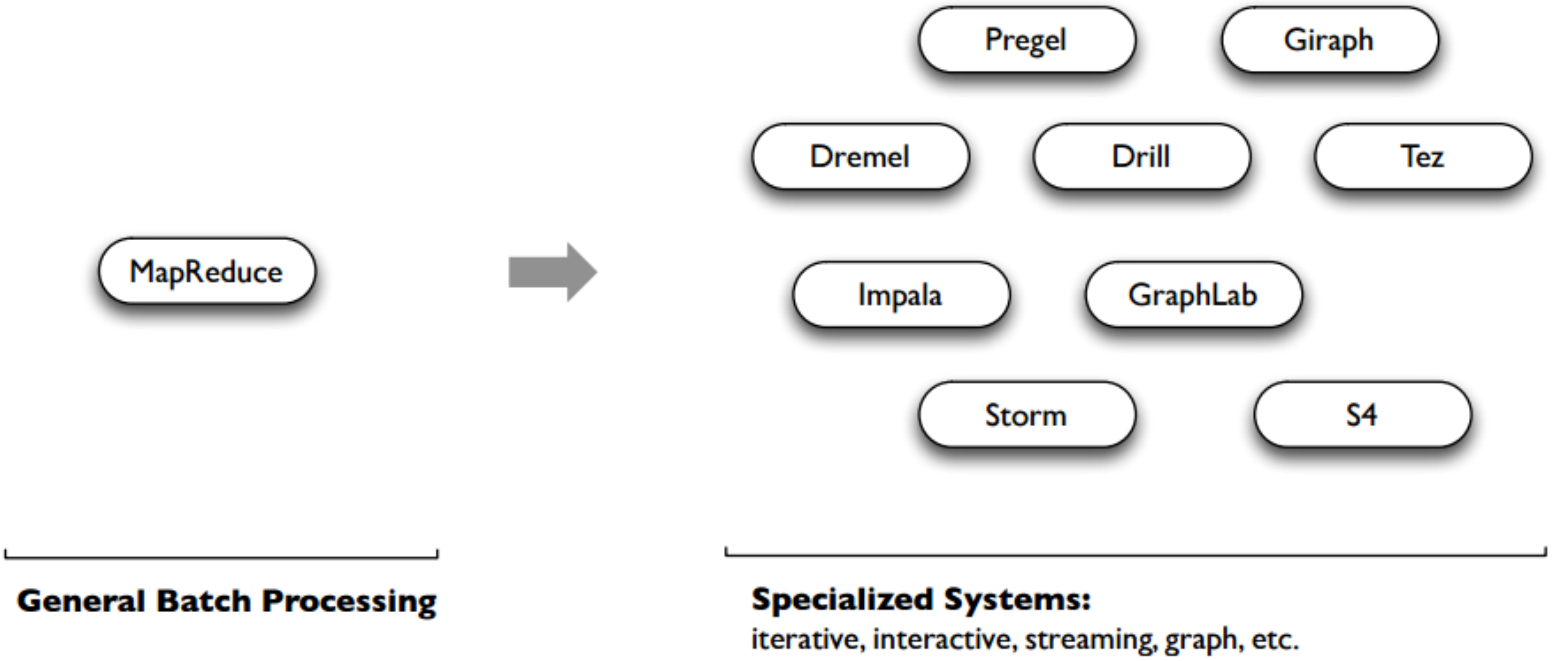
Performancia

Daytona Gray Sort 100TB Benchmark

	Data Size	Time	Nodes	Cores
Hadoop MR (2013)	102.5 TB	72 min	2,100	50,400 physical
Apache Spark (2014)	100 TB	23 min	206	6,592 virtualized

source: http://databricks.com/blog/2014/11/05/spark-officially-sets-a-new-record-in-large-scale-sorting.html

Speciális eszközök



RDD – Resilient Distributed Dataset

- Egyszerűen:
  - az RDD egy kollekciója az adatpartícióknak, amelyek a worker node memóriájában vannak tárolva
- Komplexen:
  - egy interface az adat transzformációhoz
  - átmenetileg tárolja az adatokat
  - nem módosítható

RDD

- Az egyetlen lehetőség az adatok módosítására Spark-ban

- Két osztály:
  - Transformations
  - Actions

Transformations and Actions

Transformations

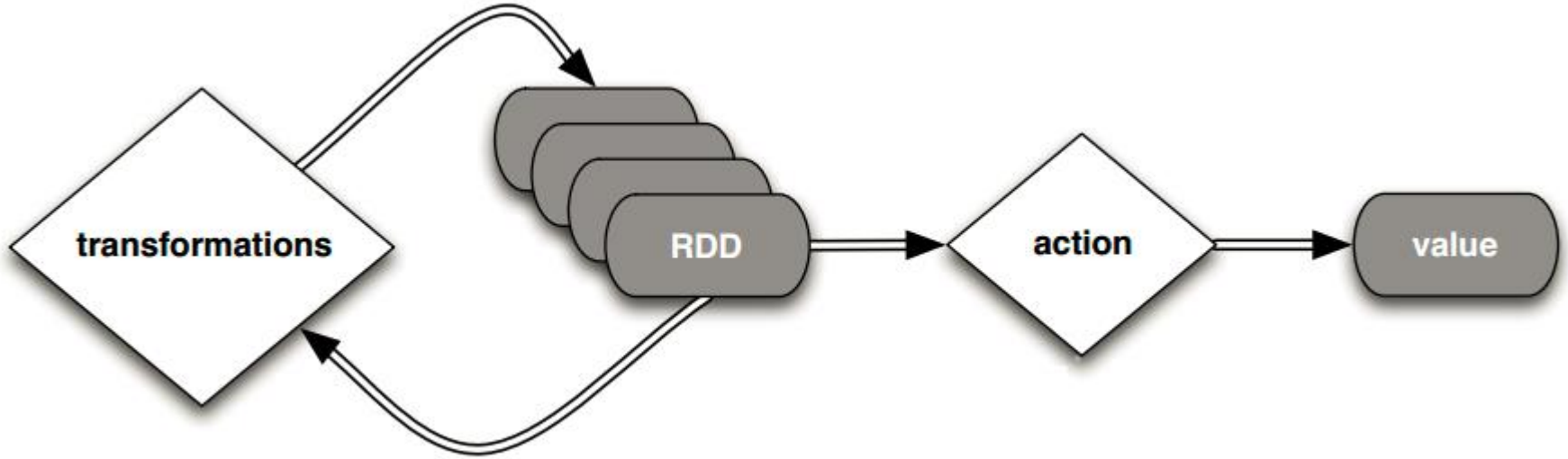
map (func)
flatMap(func)
filter(func)
groupByKey()
reduceByKey(func)
mapValues(func)
sample(...)
union(other)
distinct()
sortByKey()
...

Actions

reduce(func)
collect()
count()
first()
take(n)
saveAsTextFile(path)
countByKey()
foreach(func)
...

RDD – Lusta kiértékelés

- A transformációk, csak a metadata-t fogják változtatni



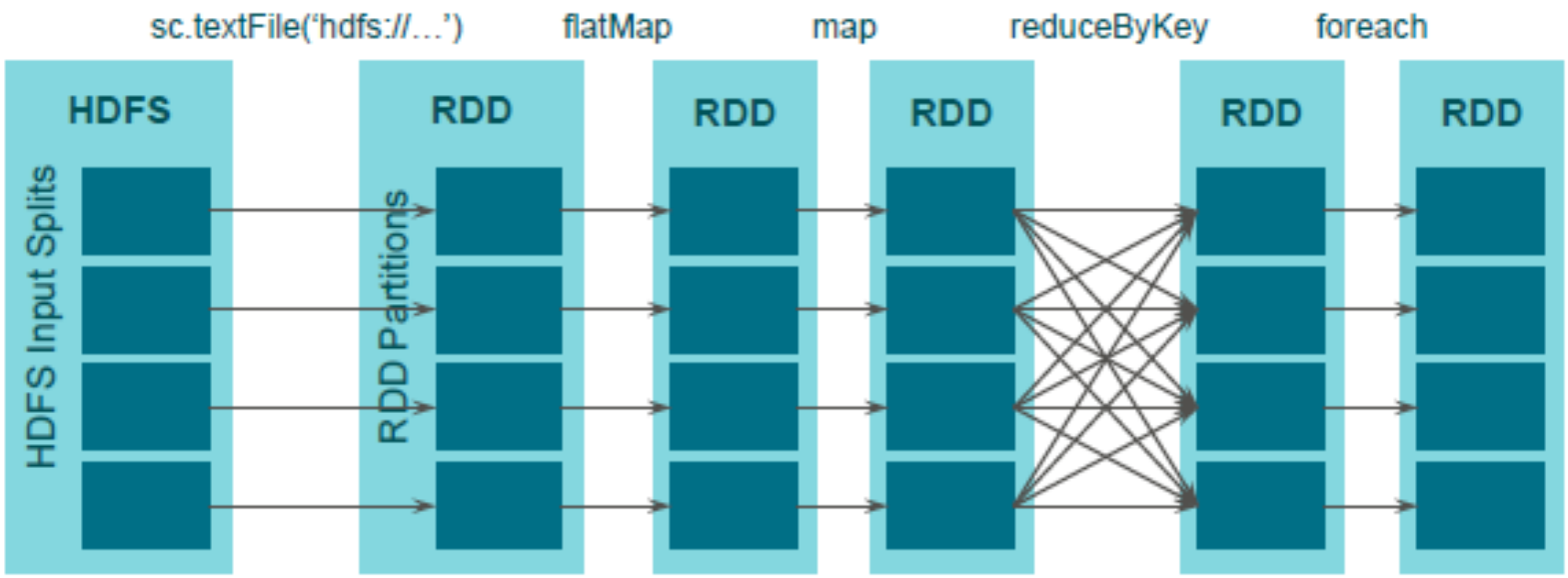
RDD Metadata

- Metainformációk
  - Partitions – mely adatok kapcsolódnak ehhez az RDD-hez
  - Dependencies – a „szülő” RDD-k listája
  - Compute – a „szülő” RDD-ken végezendő függvény
  - Preferred Locations – hol van a legjobb hely a számítás elvégzésére (data locality)
  - Partitioner – Hogyan vágjuk szét az adatot partíciókba

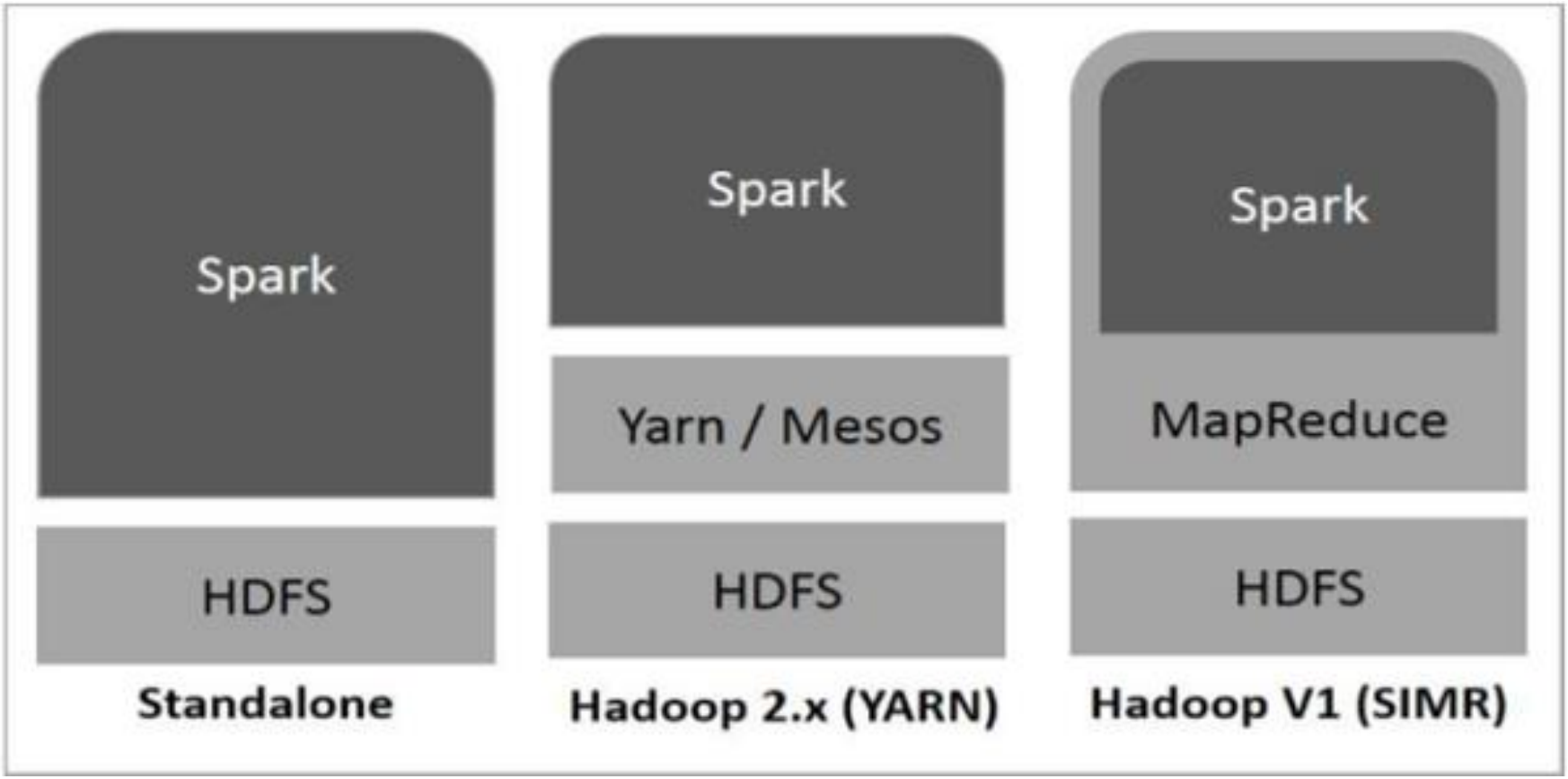
WordCount example

```
def printfunc (x):
    print 'Word "%s" occurs %d times' % (x[0], x[1])

infile = sc.textFile('hdfs://sparkdemo:8020/sparkdemo/textfiles/README.md', 4)
rdd1 = infile.flatMap(lambda x: x.split())
rdd2 = rdd1.map(lambda x: (x, 1)).reduceByKey(lambda x, y: x+y)
print rdd2.toDebugString()
rdd2.foreach(printfunc)
```

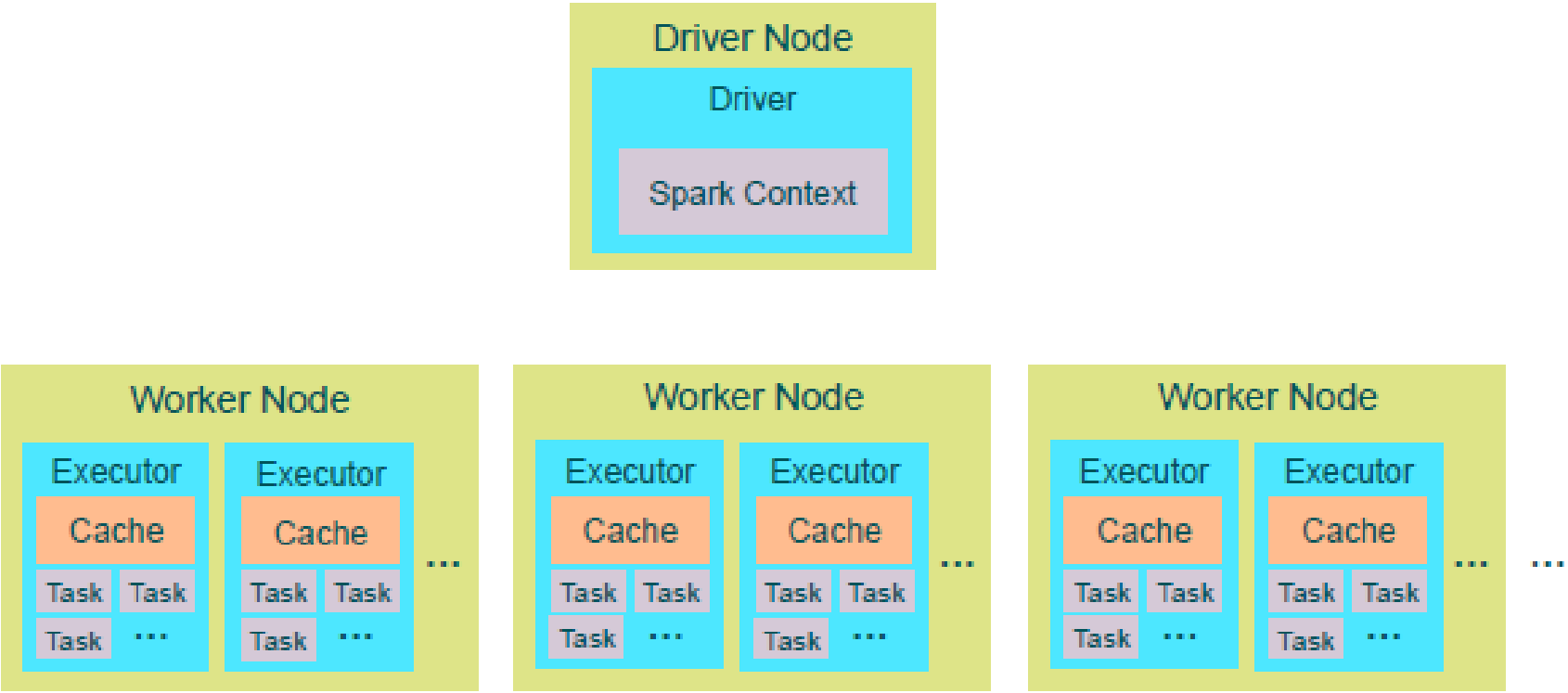


Futatási környezetek



Spark Architektúra

- Cluster



Spark Cluster

- Driver
  - Spark Shell elérés (Scala, Python, R)
  - A hely ahol elkészül a SparkContext
  - Az RDD-t átalakítja egy lekérdezési gráffá
  - Stage-ekbe pakolja a lépéseket
  - Ütemezi a taskokat és íránytja a futásukat
  - Tárolja a metadata-t az RDD-kről
  - WebUI-t futtat

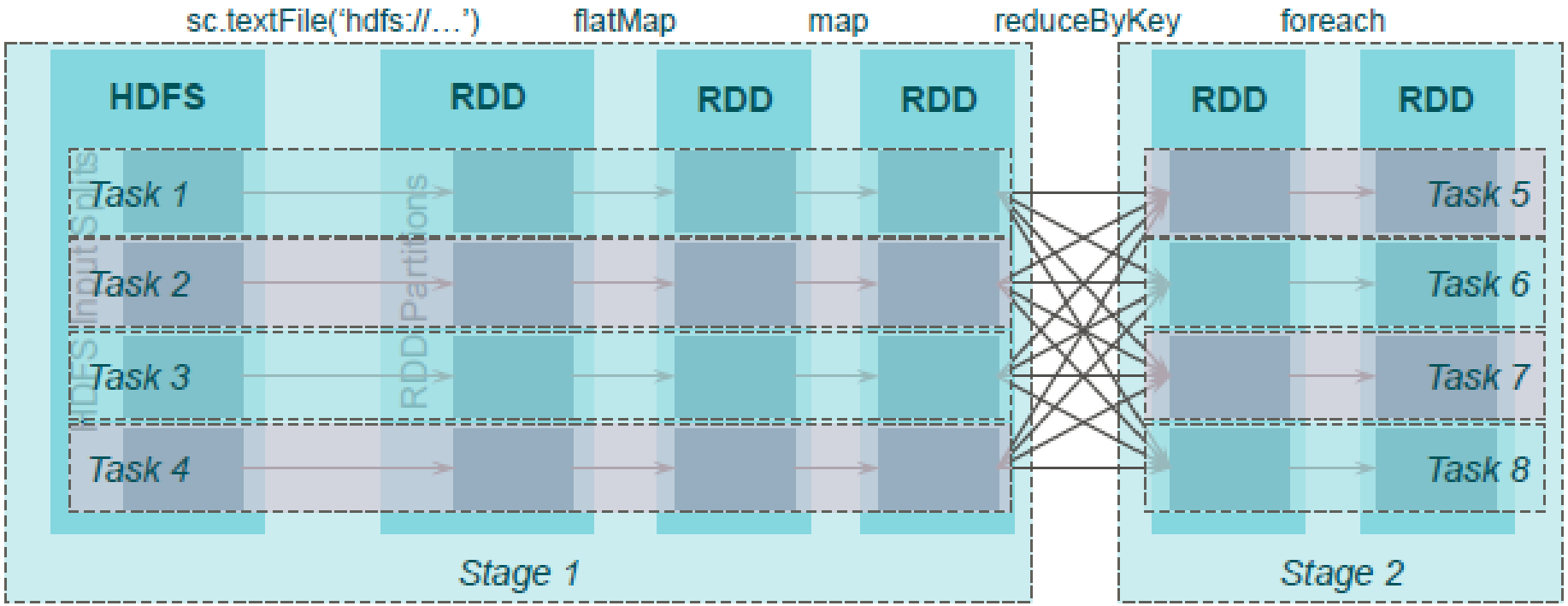
Spark Cluster

- Executor
  - Tárolja az adatot a cache-ben (JVM Heap, HDD)
  - Olvassa az adatot a külső forrásról
  - Írja az adatot külső forrásra
  - Elvégzi az adat feldolgozásokat

Alkalmazás dekompozíció

- **Alkalmazás**
  - Egy SparkContext ami tárol adatot a feldolgozáshoz, és ütemezi a jobok sorozatát, párhuzamos is lehet
- **Job**
  - RDD transzformációk sorozata, amely egy action-nel vagy adat kiírással végződik, driver alkalmazás vezérli
- **Stage**
  - Transzformációk egy olyan sorozata, amelyet egy független worker el tud végezni.
- **Task**
  - Egy stage futtatása egy adat partíción.

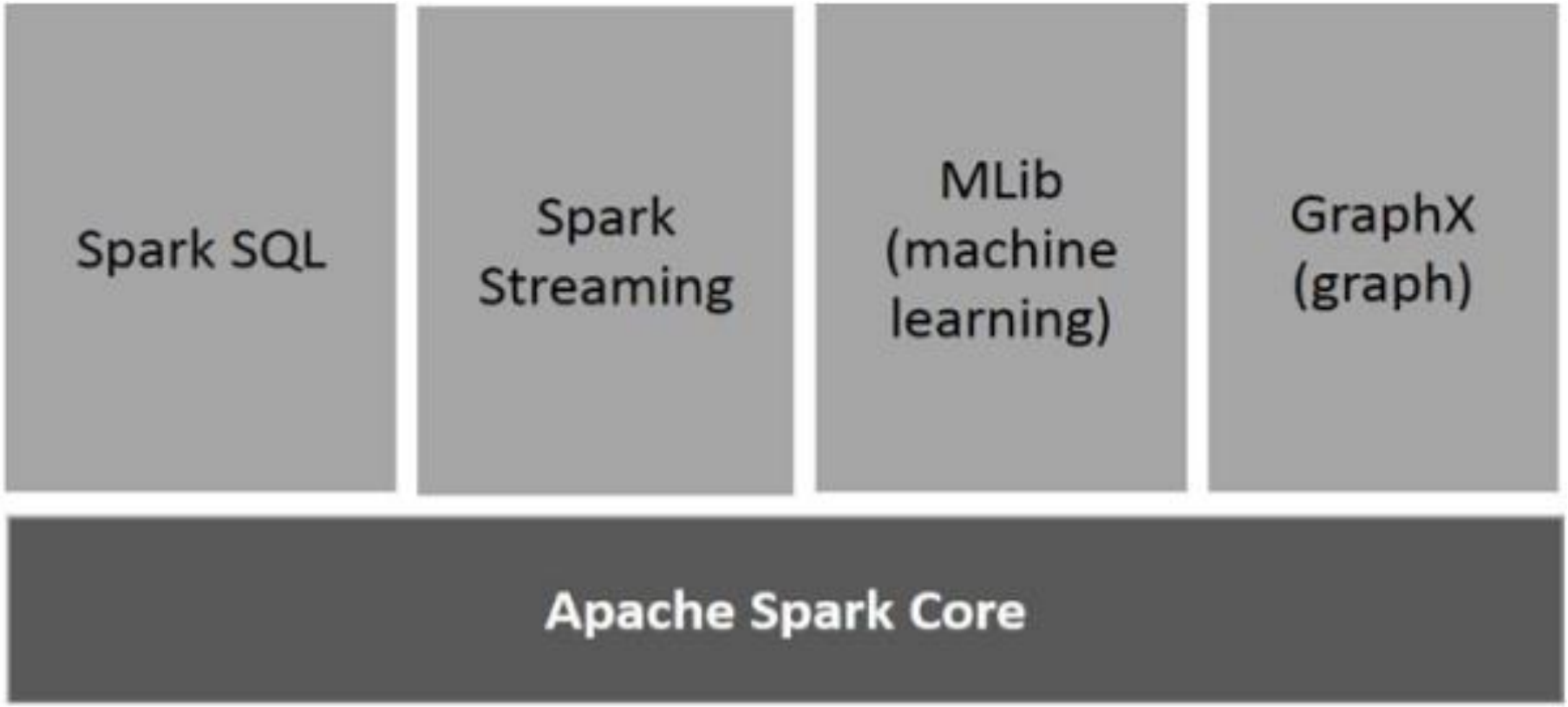
WordCount példa



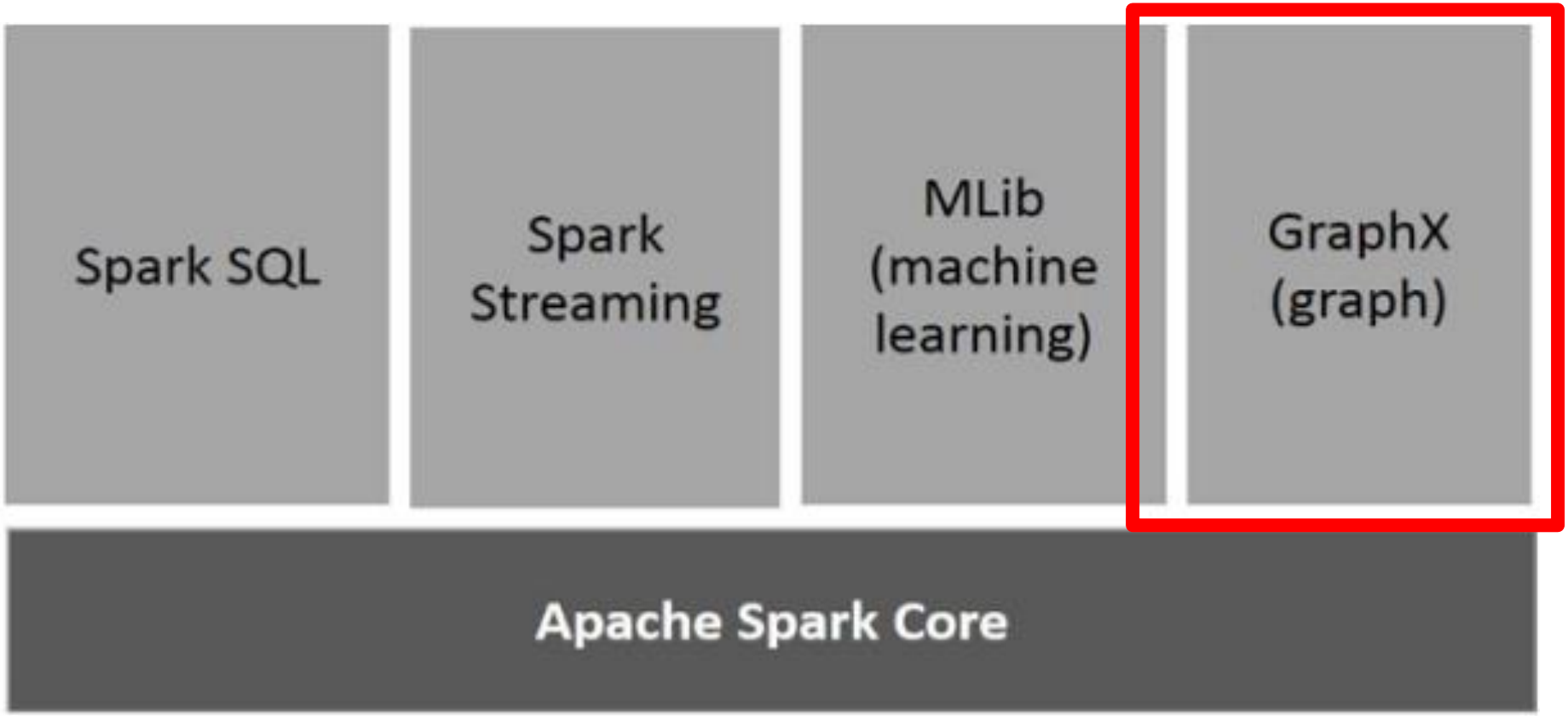
Spark perzisztencia

Persistence Level	Leírás
MEMORY_ONLY	Az RDD deszerializálzt JAVA objektumként tároljuk. Ha nem fér el a memóriában újraszámoljuk.
MEMORY_AND_DISK	Az RDD deszerializálzt JAVA objektumként tároljuk. Ha nem fér el a memóriában tároljuk a lemezen.
MEMORY_ONLY_SER	Ua. mint MEMORY_ONLY, csak szerializálva tároljuk a memóriában.
MEMORY_AND_DISK_SER	Ua. mint MEMORY_AND_DISK, csak szerializálva tároljuk a memóriában.
DISK_ONLY	Az adatokat a lemezen tároljuk.
MEMORY_ONLY_2, DISK_ONLY_2	Ua. mint feljebb, csak replikáljuk a partíciókat a klaszterben.

Spark eszközök



Spark eszközök



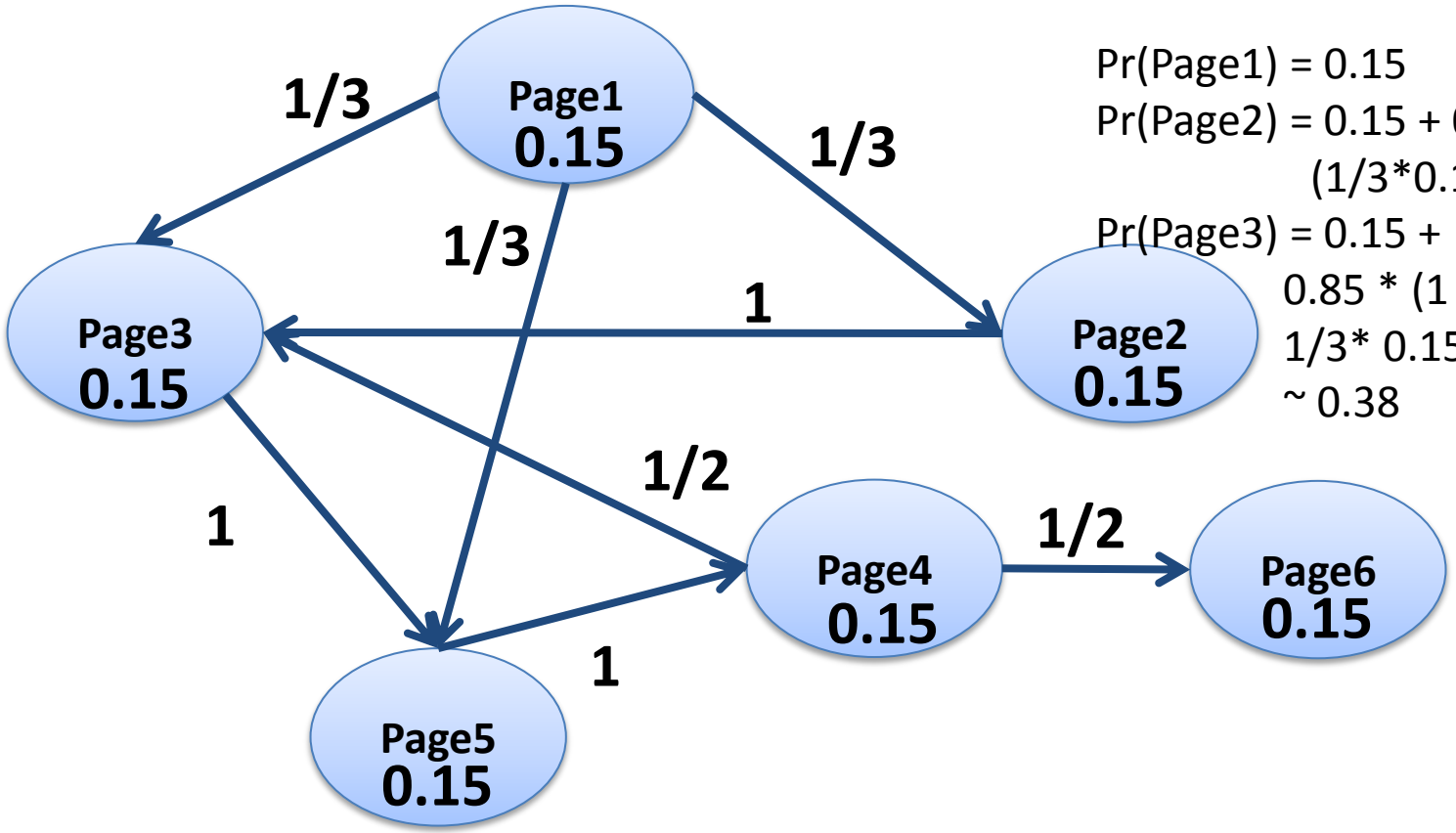
Gráf feldolgozási feladatok

- PageRank
  - Node fontosság meghatározás
- Háromszögek keresése
  - Társaságok megtalálása

PageRank példa

ResetProb: 0.15

Pr(Page) = resetProb + (1-resetProb) * szumAll(NeighPage * Edge)



Pr(Page1) = 0.15

Pr(Page2) = 0.15 + 0.85 * (1/3*0.15) ~0.19

Pr(Page3) = 0.15 + 0.85 * (1 *0.15 + 1/3* 0.15 + ½ * 0.15) ~ 0.38

Kihívások

- Gráfok nagyok
  - Milliós nagyságrendű csúcsok és élek
- Tárolás, feldolgozás egy serveren:
  - Memória limit
  - Lassú
- Tárolás elosztva:
  - Sok kommunikáció a gépek között

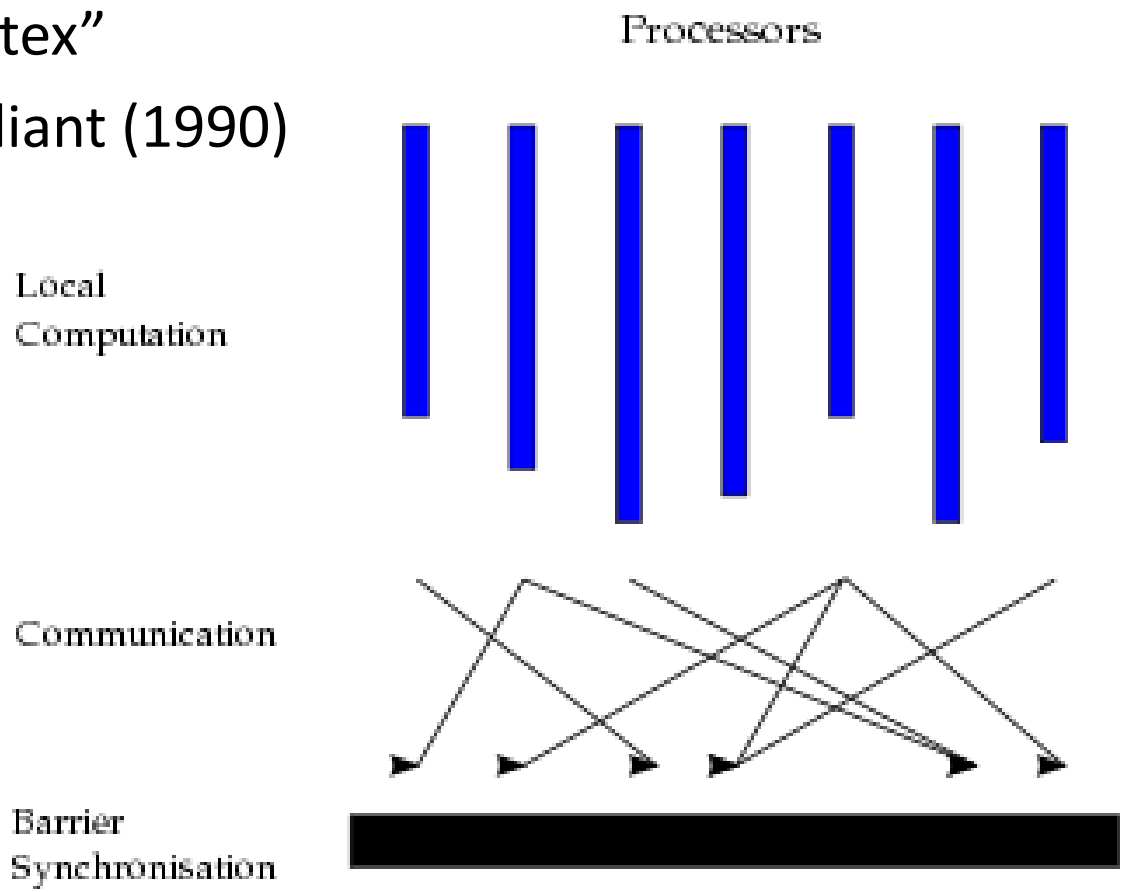
Tipikus gráf feldolgozás

- Iteratív
- Minden csúcs rendelkezik értékkel
- Minden iterációban:
  - Újraszámolja a saját értékét a szomszédok és az élek alapján

- Iteráció vége:
  - Fix számú iteráció
  - Nincs új érték a gráfban

Bulk Synchronous Parallel(BSP) Model

- „Think like a vertex”
- Originally by Valiant (1990)

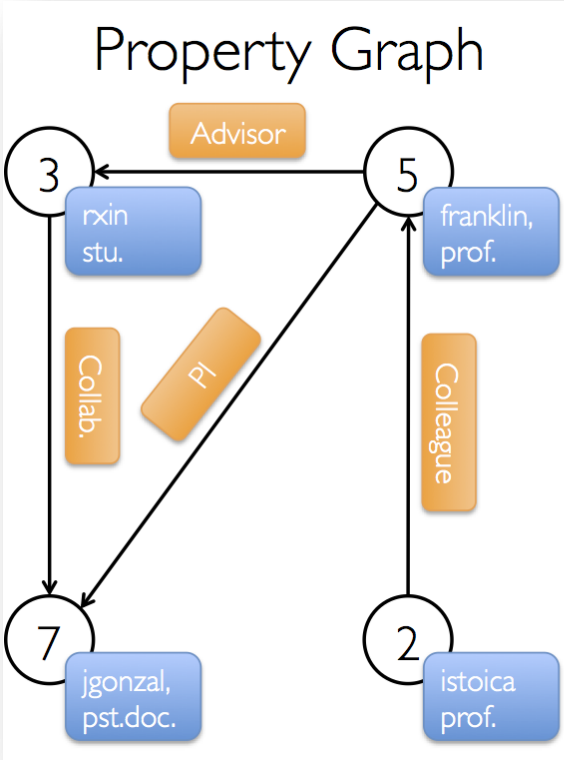


GraphX

- Optimális particionálás és indexelés a csúcsok és élek tárolására
- Alapstruktúra:
  - Property graph
- API
  - Bejárás
  - Elérés
- Alap függvények
  - PageRank
  - Connected components
  - Tringle counting

Property graph

- Felhasználói struktúrák a csúcsokhoz és élekhez



Vertex Table

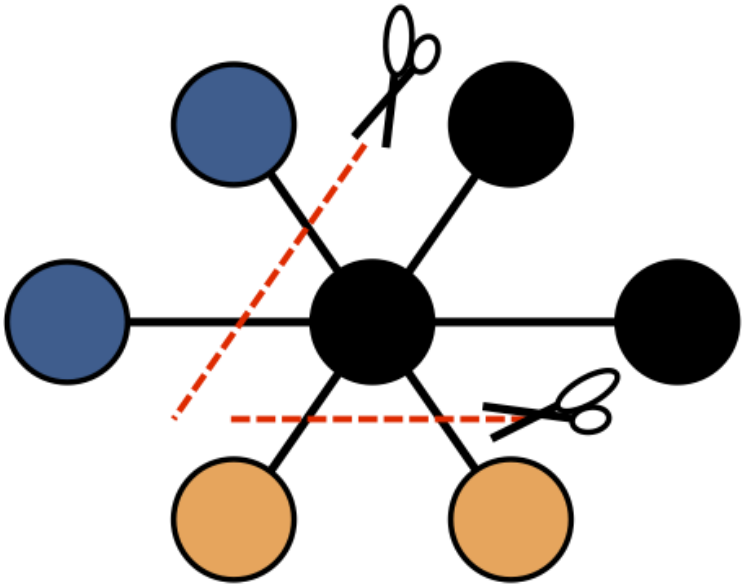
Id	Property (V)
3	(rxin, student)
7	(jgonzal, postdoc)
5	(franklin, professor)
2	(istoica, professor)

Edge Table

SrcId	DstId	Property (E)
3	7	Collaborator
5	3	Advisor
2	5	Colleague
5	7	PI

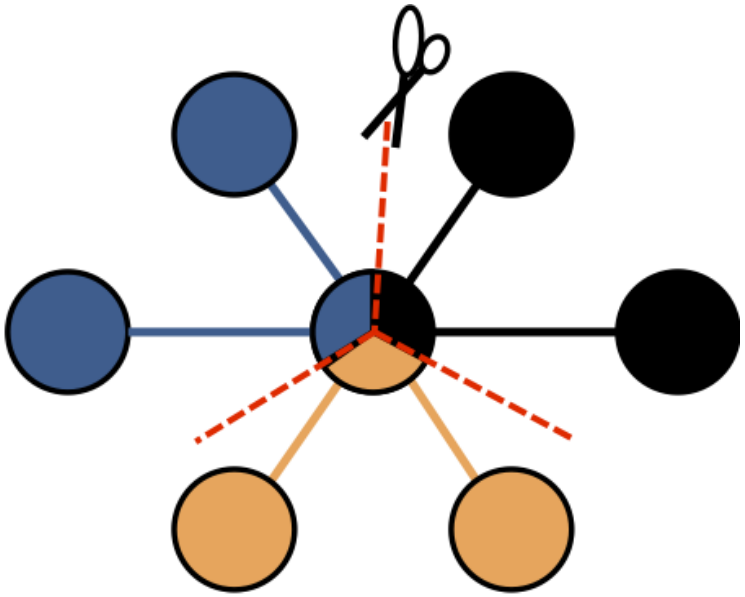
Optimális gráftárolás

Általában



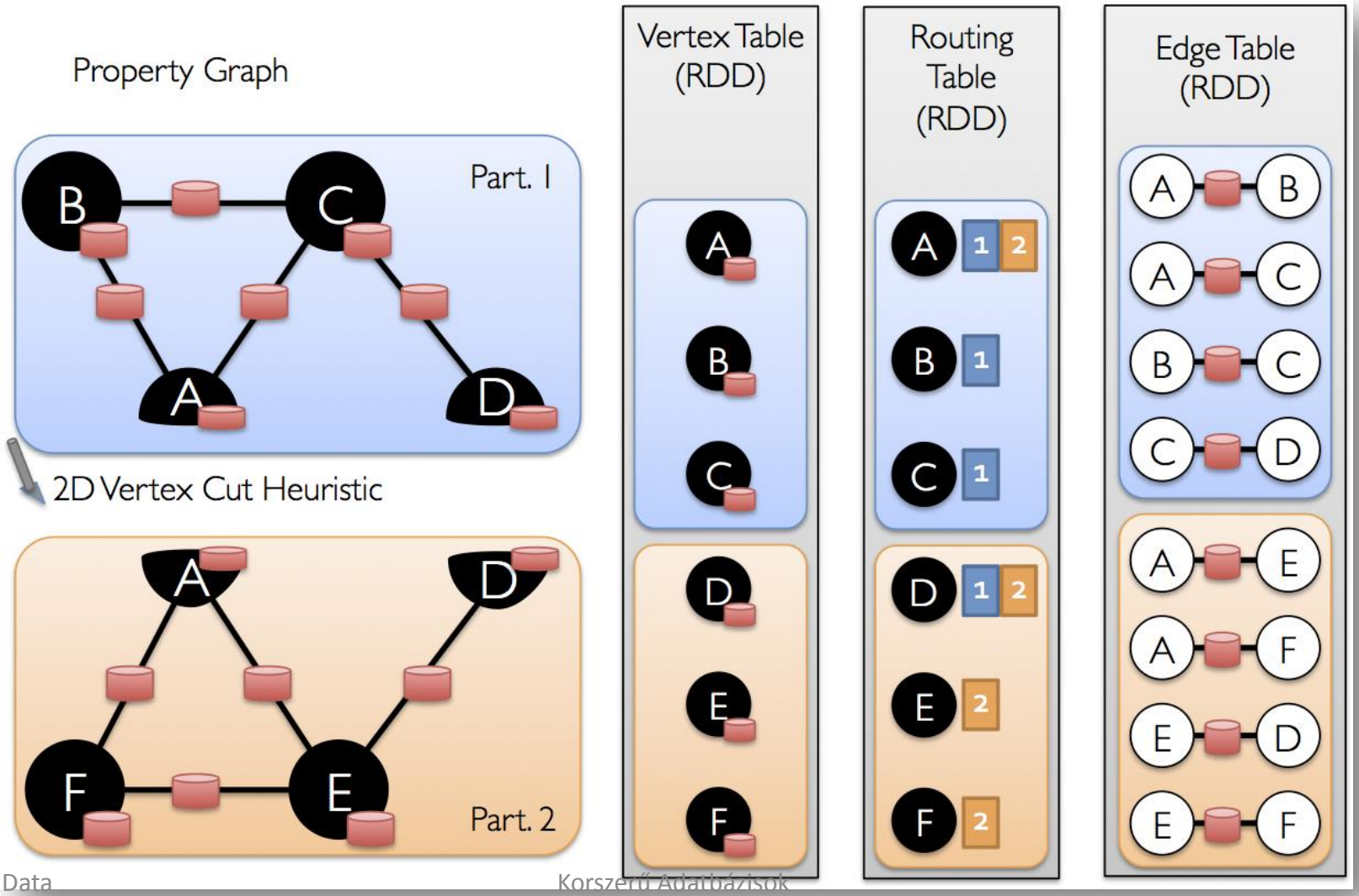
Edge Cut

GraphX



Vertex Cut

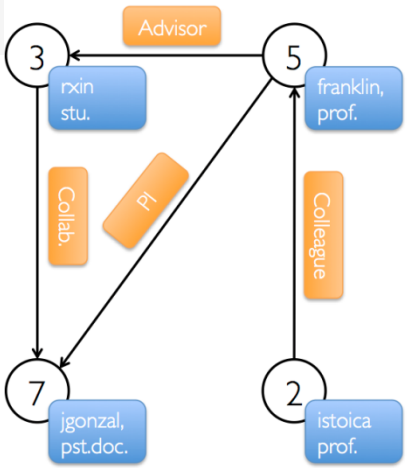
Optimális adattárolás



GraphX

```scala
// Assume the SparkContext has already been constructed
val sc: SparkContext
// Create an RDD for the vertices
val users: RDD[(VertexId, (String, String))] =
  sc.parallelize(Array((3L, ("rxin", "student")), (7L, ("jgonzal", "postdoc")),
                       (5L, ("franklin", "prof")), (2L, ("istoica", "prof"))))
// Create an RDD for edges
val relationships: RDD[Edge[String]] =
  sc.parallelize(Array(Edge(3L, 7L, "collab"),    Edge(5L, 3L, "advisor"),
                       Edge(2L, 5L, "colleague"), Edge(5L, 7L, "pi")))
// Define a default user in case there are relationship with missing user
val defaultUser = ("John Doe", "Missing")
// Build the initial Graph
val graph = Graph(users, relationships, defaultUser)
```

Property Graph



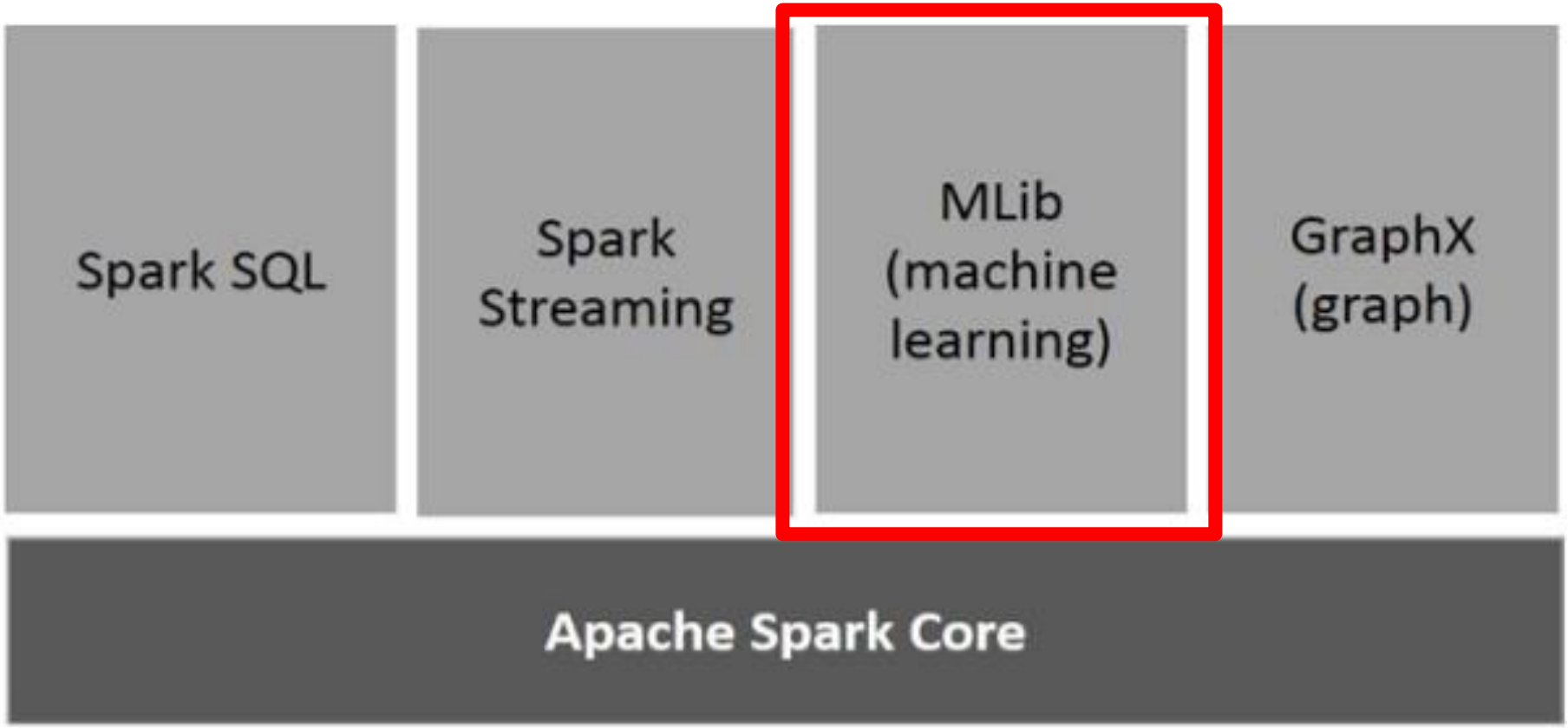
Vertex Table

Id	Property (V)
3	(rxin, student)
7	(jgonzal, postdoc)
5	(franklin, professor)
2	(istoica, professor)

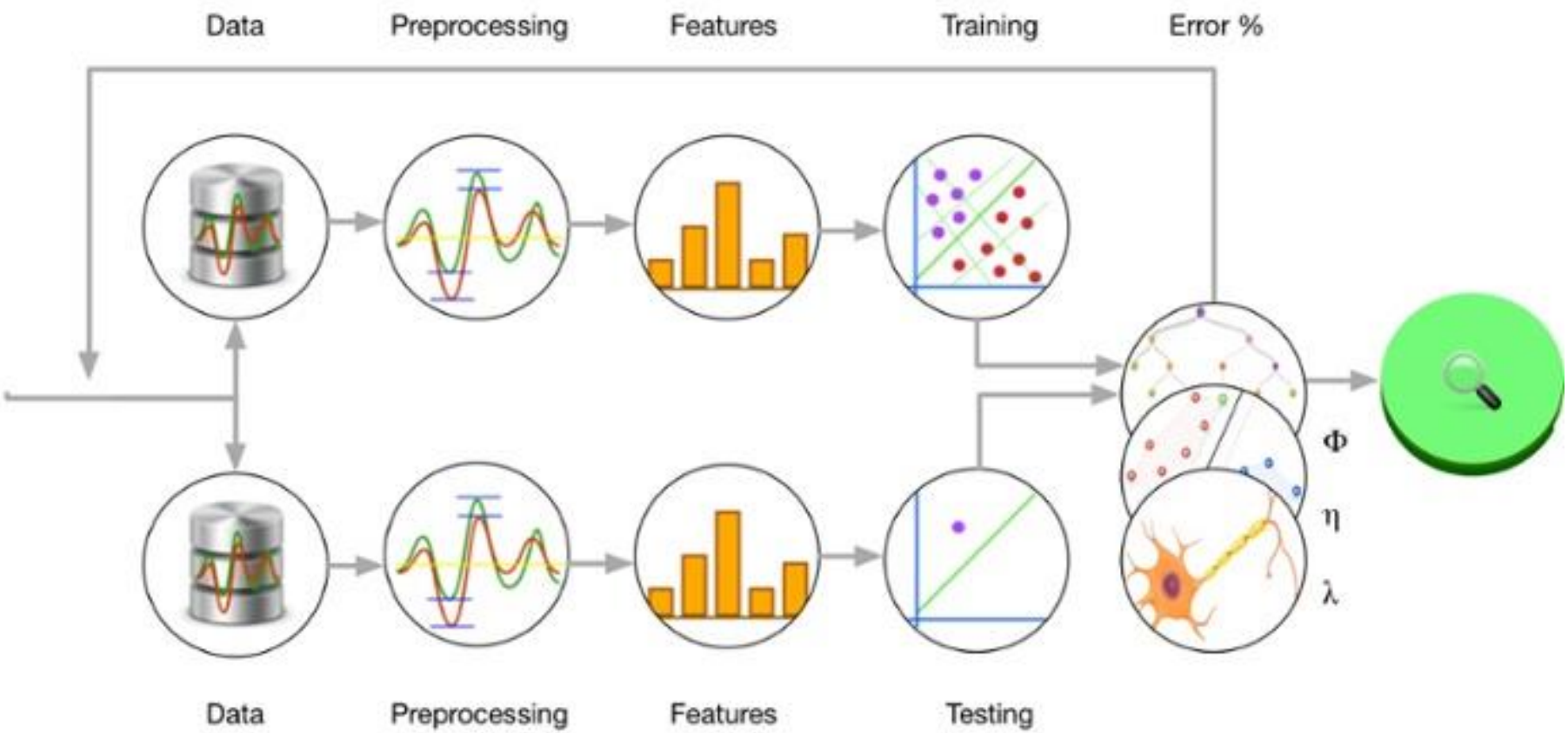
Edge Table

SrcId	DstId	Property (E)
3	7	Collaborator
5	3	Advisor
2	5	Colleague
5	7	PI

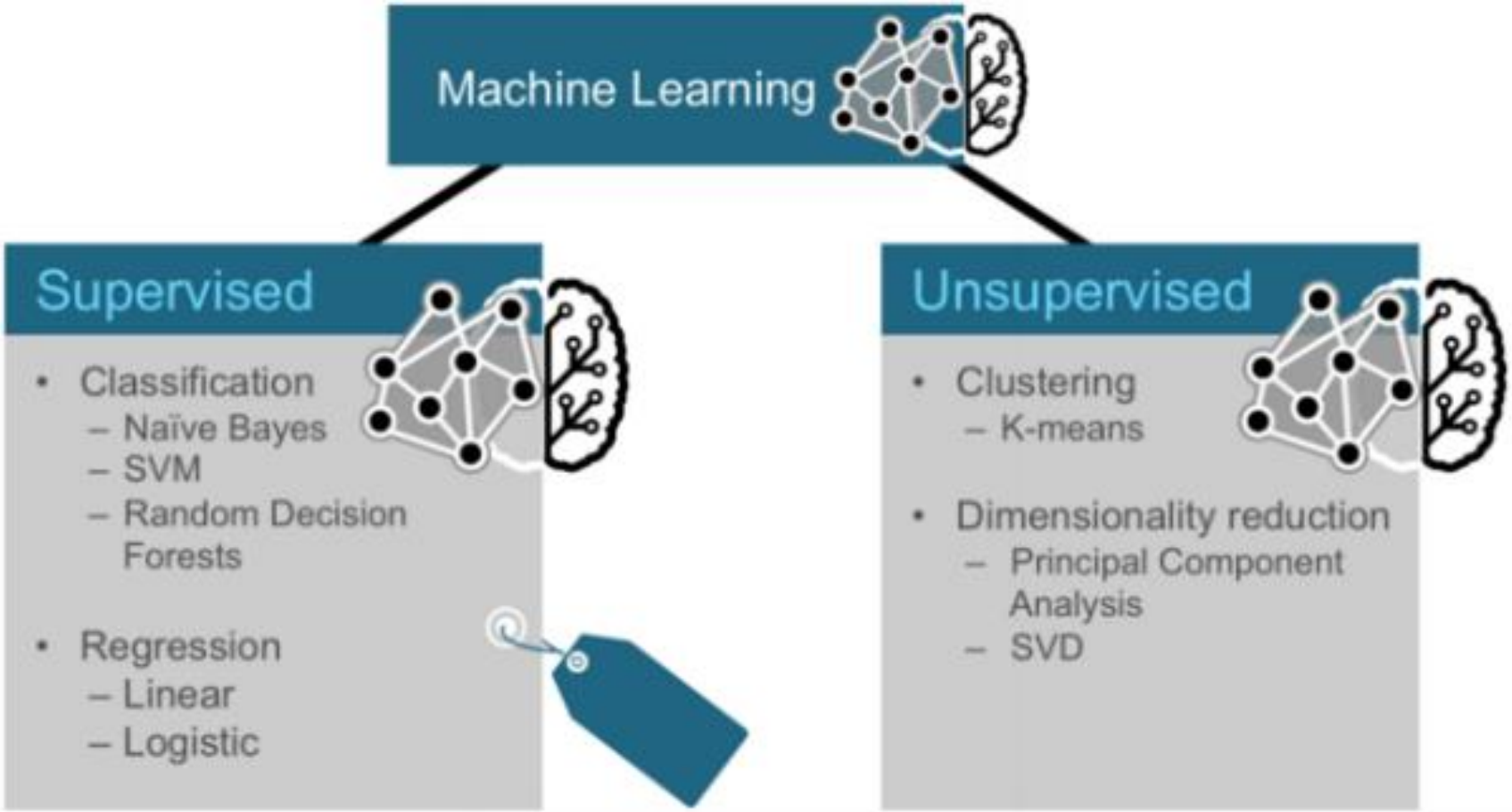
Spark eszközök



Machine Learning



Machine Learning Library (MLlib)

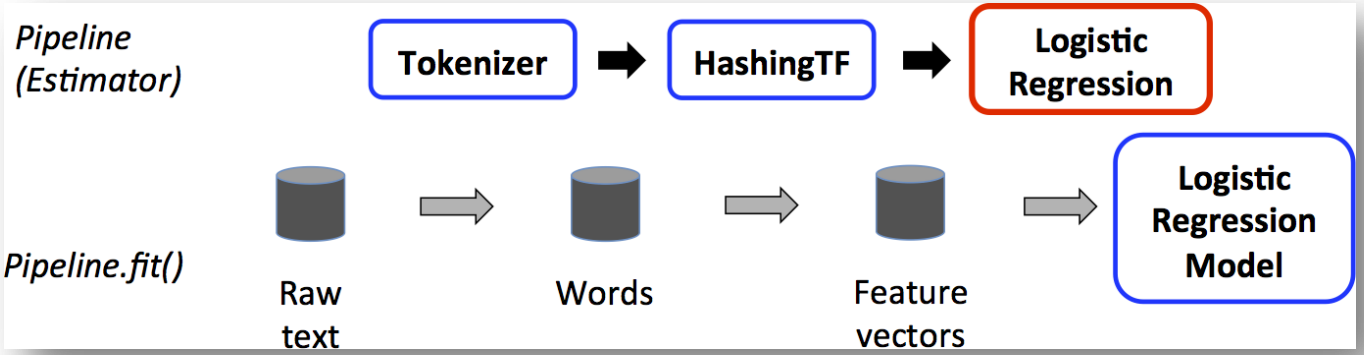


Machine Learning Library (MLlib)

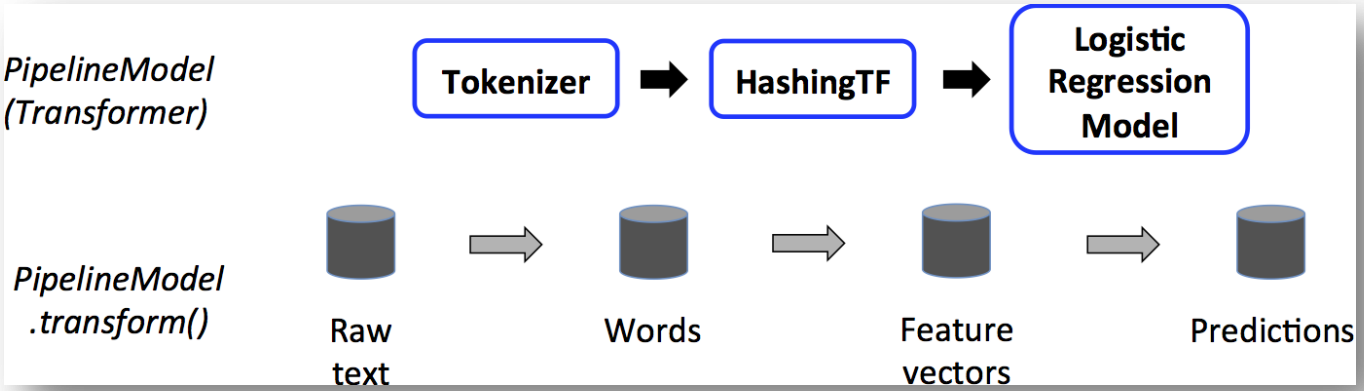
- 2 csomagban érhetőek el:
  - spark.mllib:
    - RDD-ken dolgozik
    - alacsonyabb szintű
  - spark.ml
    - DataFrame-ken dolgozik
    - magasabbszintű
    - pipeline-ok készítése

Pipeline

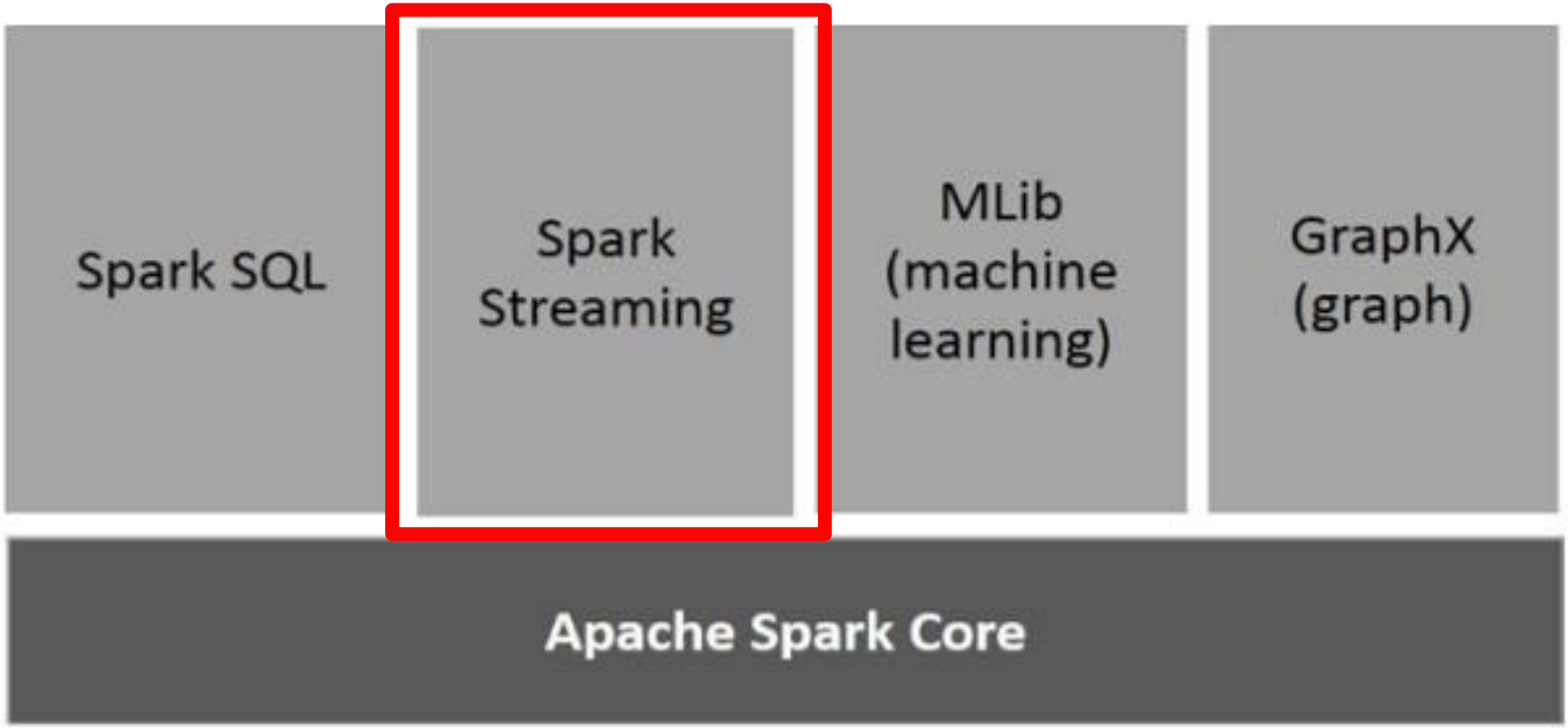
Training



Test



Spark eszközök



Spark Streaming

statikus adat

sok adat másodpercenként

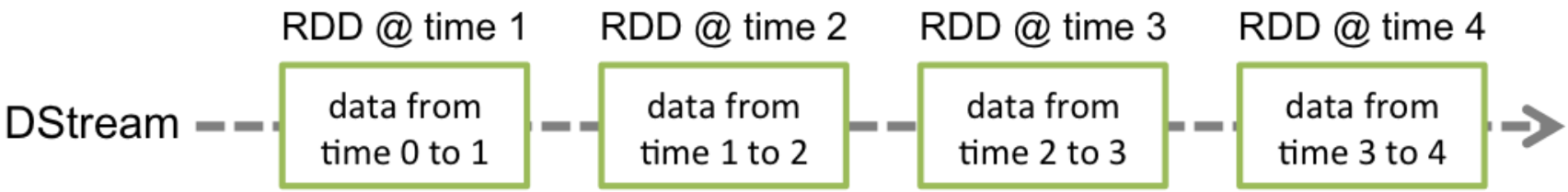
Alkalmazás	Szenzor	Web	Mobil
Behatolás detektálás	Hiba detektálás	Oldal elemzés	Hálózati mérések elemzése
Csalás detektálás	Mérési adat feldolgozás	Ajánlások	Hely alapú reklámok
Log elemzés	….	Érzelem elemzés	….

Spark Streaming

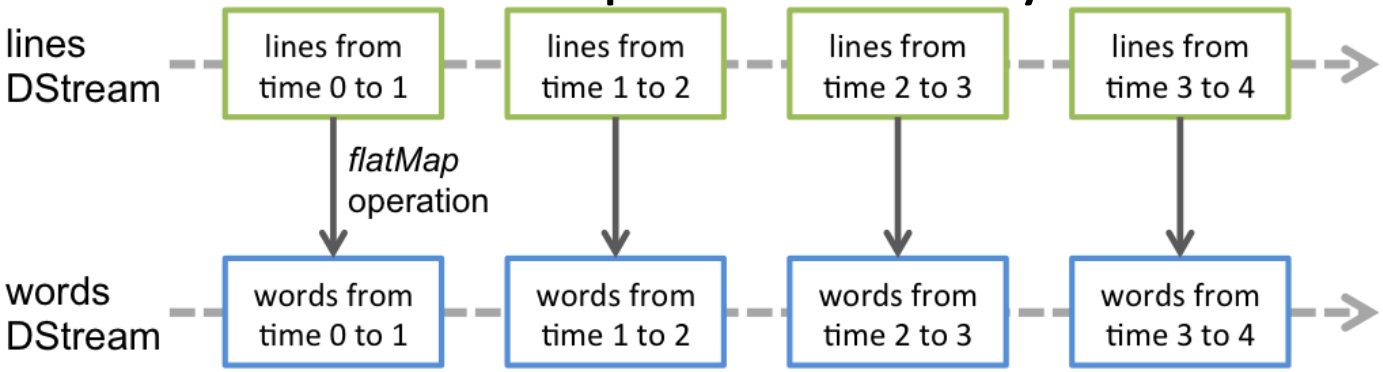
- Bementi stream darabolása
- Batch folyamat futtatása a darabokon
  - lehet: graphX, sparkML
- Kimenet tárolása

Discretize Stream (DStreams)

- DStream: RDD-k folytonos sorozata



- Minden DStream-en operátor érvényes az RDD-n

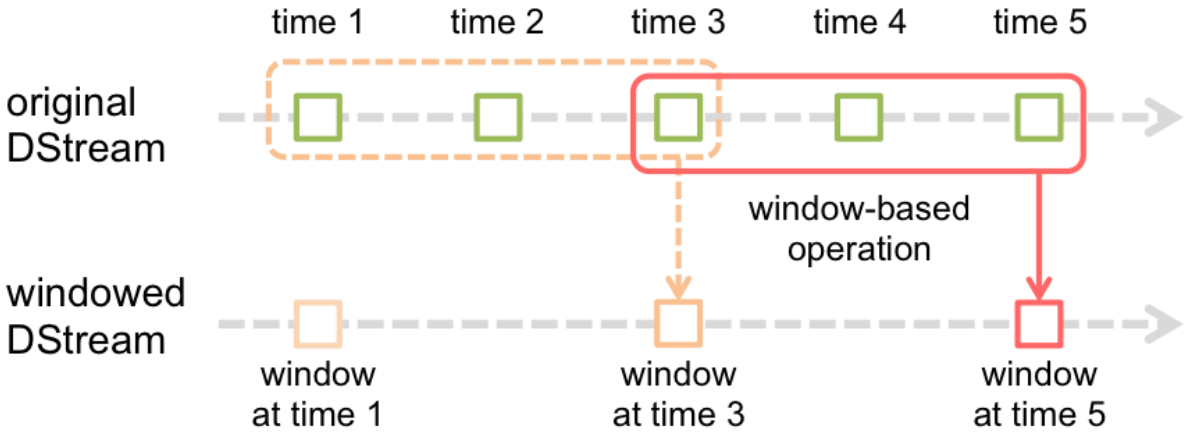


Input Dstreams

- Két típusú bemenet:
  - alap források: File system, Socket connection,…
  - fejlettebb források: Kafka, Flume, Kinesis, Twitter,…

Window operátorok

- Lehetőség csúszó ablakos számításokra.
- Paraméterek:
  - ablak hossz (pl.: 3)
  - csúszási intervallum (pl: 2)



Window operátorok

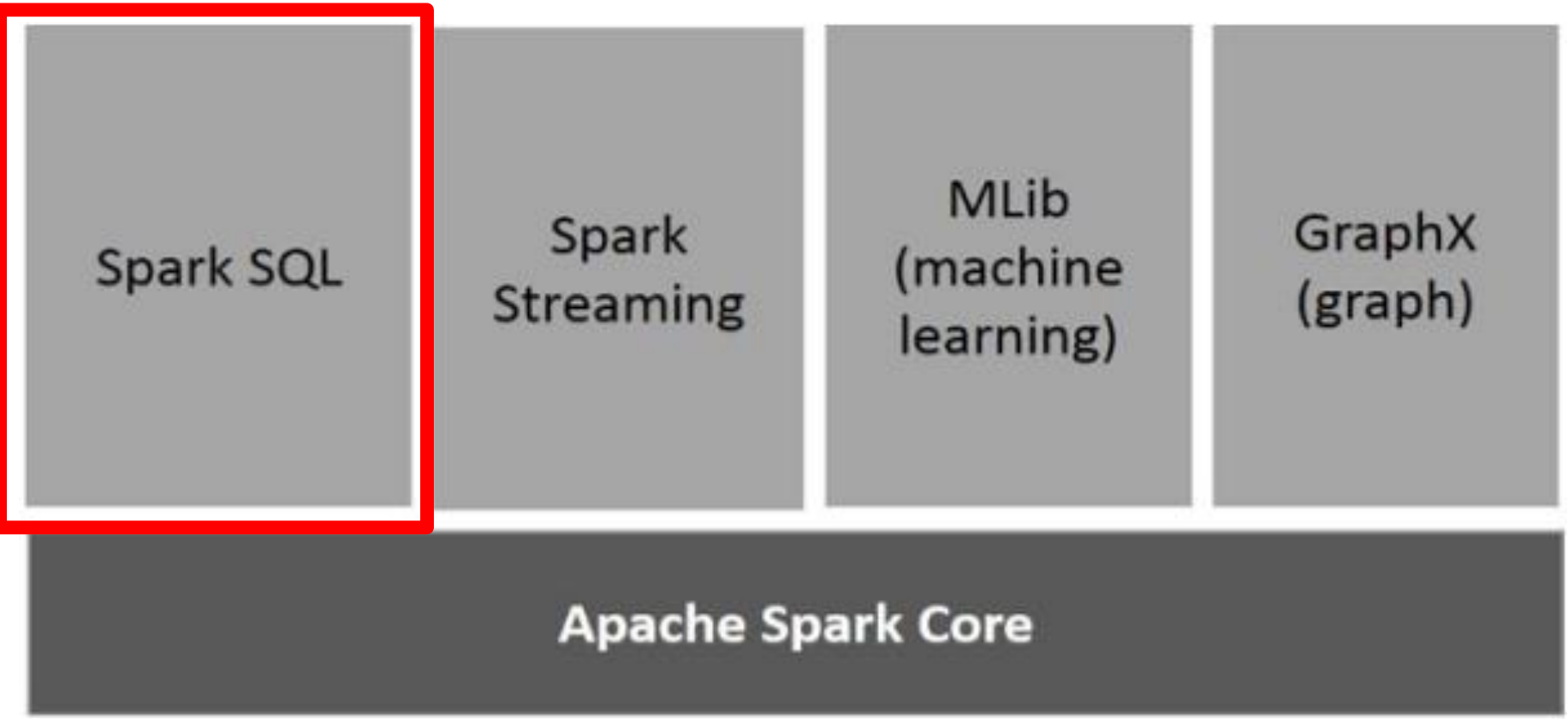
- példa: Az utolsó 30 másodpercben akarjuk a szavak számát megszámolni minden 10 másodpercben.

```
val windowedWordCounts = pairs.reduceByKeyAndWindow(
            (a:Int,b:Int) => (a + b), Seconds(30), Seconds(10)
)
```

Checkpointing

- Az adatok folyamatosak, így hiba esetén nincs visszaállítási lehetőség
- A spark-nak checkpointokra van szüksége, hogy visszatudjon állítani egy korábbi állapotot
- Típusai:
  - Metadata szintű: a stream számításhoz szükséges információkat tárolja (driver hibákra jó)
  - Adat szintű: korábbi RDD adatok tárolása (statefull adat hibákra jó)
- Checkpoint beállítása:
  - egy fault-tolerant rendszeren mappa megadása (pl HDFS)
  - ssc.checkpoint(checkpointDirectory)

Spark eszközök



Spark SQL

- Apache Spark modulja strukturált adatok kezelésére
- Keverni lehet a Spark programot és az SQL-t
- Több féle adatforrás ugyanazon a módon (Hive, Avro, Parquet, JSON, JDBC)
- Alapja a DataFrame-k:
  - Elosztott adattípus (RDD-n alapszik)
  - Nevesített oszlopokkal rendelkezik
- Rendelkezik lekérdezés optimalizációval

Dataframe létrehozása

- Szükség van egy SparkSession-re, amely a Spark funkcionalitást biztosítja!

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.getOrCreate()

df = spark.read.json("examples/src/main/resources/people.json")
# Displays the content of the DataFrame to stdout
df.show()
```

DataFrame műveletek

- Dataframe séma lekérdezése

```
df.printSchema()
# root
# |-- age: long (nullable = true)
# |-- name: string (nullable = true)
```

- Egyszerű Select lekérdezése

```
df.select("name").show()
# +-------+
# |   name|
# +-------+
# |Michael|
# |   Andy|
# | Justin|
# +-------+
```

DataFrame műveletek

- Számított oszlop lekérdezése (age+1)

```
df.select(df['name'], df['age'] + 1).show()
# +-------+---------+
# |   name|(age + 1)|
# +-------+---------+
# |Michael|     null|
# |   Andy|       31|
# | Justin|       20|
# +-------+---------+
```

- Szűrés használata

```
df.filter(df['age'] > 21).show()
# +---+----+
# |age|name|
# +---+----+
# | 30|Andy|
# +---+----+
```

DataFrame műveletek

- Csoportosítás, count

```
df.groupBy("age").count().show()
# +----+-----+
# | age|count|
# +----+-----+
# |  19|    1|
# |null|    1|
# |  30|    1|
# +----+-----+
```

SQL lekérdezések használata

- Regisztráljuk a DataFrame-t, mint egy View
  - Ez átmeneti, session végén megszűnik

```
df.createOrReplaceTempView("people")
```

- Select futtatása

```
sqlDF = spark.sql("SELECT * FROM people")
sqlDF.show()
# +----+-------+
# | age|   name|
# +----+-------+
# |null|Michael|
# |  30|   Andy|
# |  19| Justin|
# +----+-------+
```

SQL lekérdezések használata

- Datafame globális regisztrációja, hogy minden session lássa

```
df.createGlobalTempView("people")

spark.sql("SELECT * FROM global_temp.people").show()
# +----+-------+
# | age|   name|
# +----+-------+
# |null|Michael|
# |  30|   Andy|
# |  19| Justin|
# +----+-------+
```

```
spark.newSession()\
        .sql("SELECT * FROM global_temp.people").show()
# +----+-------+
# | age|   name|
# +----+-------+
# |null|Michael|
# |  30|   Andy|
# |  19| Justin|
# +----+-------+
```

Dataframe létrehozása

- Row típusok RDD-jéből DataFrame készítése

```
from pyspark.sql import Row

sc = spark.sparkContext

lines = sc.textFile("examples/src/main/resources/people.txt")
parts = lines.map(lambda l: l.split(","))
people = parts.map(lambda p: Row(name=p[0], age=int(p[1])))

schemaPeople = spark.createDataFrame(people)
```

DataFrame készítés sémával

```
from pyspark.sql.types import StringType, StructType, StructField

sc = spark.sparkContext

lines = sc.textFile("examples/src/main/resources/people.txt")
people = []
for l in lines:
    p = l.split(",")
    people.append((p[0], p[1].strip()))

schemaString = ["name","age"]

fields = [StructField(field_name, StringType(), True) for field_name in schemaString]
schema = StructType(fields)

schemaPeople = spark.createDataFrame(people, schema)
```

Köszönöm a figyelmet!